Scripting-Python Foundation (61549742)

Company Name
Capgemini India Technology services Pvt Ltd

Employee Name
aryan.b.raj

Date of Attempt
18-Dec-2025

Employee ID
30080136

Index

Score Analysis

Your scores, a quick overview of your performance and your overall percentage.

Section Score Analysis

A quick overview of sectional performance along with percentages.

Section Skill Analysis

An overview of your proficiency in specific skills.

Individual Development Plan - IDP

Focus on your strengths and the areas of improvement, along with developmental tips to work on.

Difficulty Level Analysis

A comprehensive insight into the candidate's performance at 3 difficulty levels.

Proctoring Analysis

A quick overview of the proctoring-related aspects of the assessment.

Test Log

A quick overview of the test status, timestamp, and recorded IP address.

Question Details

An overview of each question and the candidate's response, offering a thorough assessment of their performance.

Disclaimer

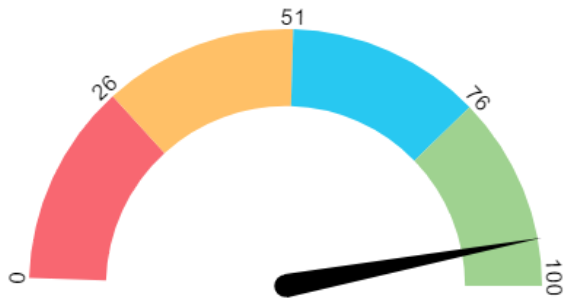
Disclaimer on subjective customised assessments.

Score Analysis

Score: 71/ 75 **Time Taken: 58 min 10 sec / 120 min**

Note: Candidates answering in Section Switch mode receive an extra 5 minutes to allow time for navigating between sections, reviewing, and revising their answers.

Proficient (95%)



Beginner	Intermediate	Experienced	Proficient
(0% - 25%)	(26% - 50%)	(51% - 75%)	(76% - 100%)

aryan.b.raj scored **95%** and completed assessment in **48%** of the alloted time

Test Navigation Mode: Section Switch

Section Score Analysis

Section	Percentage
Python Coding	61/65 (94%)
Exception Handling Coding	10/10 (100%)

Section Skill Analysis

Section 1: Python Coding

Total Score: **61/ 65** Negative Points: **0** Time Taken: **47 min 58 sec**

Question Analysis:

Total Question: **5** Correct: **5** Wrong: **0** Skipped: **0** Not Answered: **0**

Skills	#Questions	Skill Score
Coding - Easy	3	30/30
Coding - Medium	1	15/15
Coding - High	1	16/20

Capgemini

Section 2: Exception Handling Coding

Total Score: **10/ 10** Negative Points: **0** Time Taken: **9 min 39 sec**

Question Analysis:

Total Question: **1** Correct: **1** Wrong: **0** Skipped: **0** Not Answered: **0**

Skills	#Questions	Skill Score
Exception Handling Coding	**1**	**10/10**

Identification of strengths and skill improvement needs

Strength

Congratulations! We have identified **Programming, Exception Handling** as your strengths.

Improvement area

Congratulations! You have performed well and shown excellence in these skills. However, you will become stronger when you know your weaknesses. So go ahead and **take another skill assessment to identify your areas of improvement.**

A guide to get started on your Individual Development Plan (IDP) :

Difficulty Level Analysis

Level	Number of Questions	Correct Attempts	Correctness
Easy	3	3	100%
Medium	2	2	100%
Hard	1	1	100%

Proctoring Analysis

Images Captured: 116 **Image Violations: 0**

Image violations detected, within tolerable limit.

Multiple Faces Detected	**0**
No Face Detected	**0**
Unrecognized Face Detected	**0**

Note: The total violations are based on the custom violation settings for this test. The number of consecutive images considered as one violation is configured for all the categories (Unrecognized Face, Multiple Faces, No Face) and may differ from the default settings.

Window Violation: 0 **Time Violation: 0 min**

Time Flag: Early Finisher Flag

This candidate completed the test in 58 min 10 sec which is significantly faster than the average test time of 84 min 3 sec.

Test Log

Test Status	Date & Time	Captured IP address
Appeared On	18 Dec 2025, 07:24 PM	167.103.120.126
Completed On	18 Dec 2025, 08:22 PM	167.103.120.126
Report Generated On	18 Dec 2025, 08:23 PM	167.103.120.126

Question Details

Question: **#1**	Type: **Coding**	Skill: **Coding - Easy**	Status: **Answered**
Result: **Correct**	Level: **Easy**	Time Taken: **4 min 18 sec**	Average Time: **10 min 14 sec**
Score: **10 / 10**	Window Violation: **0 times**	Time Violation: **0 sec**	

Question #1

Triple Factor

Mark wants to find the triple factorial of a given number ***N***.

A triple factorial of a number ***N*** is the product of positive numbers less than or equal to that number and congruent to ***N*** modulo 3.

It means ***N***!!! = ***N*** * (***N***-3) * (***N***-6) * … * (x+3) * ***x***, where ***x*** is the smallest positive number which is congruent to ***N*** modulo 3.

Print the triple factorial for a given number N.

Note

The triple factorial of 0 is 1 by definition.

Since the answer can be quite large, print the answer modulo 10^9+7.

Function Description

Capgemini

In the provided code snippet, implement the provided `tripleFactorial(…)` method to print the triple factorial for a given number ***N***. You can write your code in the space below the phrase `"WRITE YOUR LOGIC HERE"`.

There will be multiple test cases running so the Input and Output should match exactly as provided.
The base Output variable `result` is set to a default value of `-404` which can be modified. Additionally, you can add or remove these output variables.

Input Format

The first line contains an integer, ***N***.

Sample Input

```
10    -- denotes N
```

Constraints

1 <= ***N*** <= 10^3

Output Format

The output contains an integer denoting the triple factorial for a given number ***N***.

Sample Output

```
280
```

Explanation

N = 10
A triple factorial of 10 is given by:
= 10!!! = 10 * (10 - 3) * (10 - 6) *…* 1 (because 1 is the smallest positive number congruent to 10 modulo 3)
= 10!!! = 10*7*4*1
= 10!!! = 280.
Hence, the output is ***280***.

Answer:

Coding Language: Python 3

Candidate Code:

```
def tripleFactorial(N):
    # this is default OUTPUT. You can change it.
    result = -404
    MOD = 10**9 + 7
    # write your Logic here:
    if N==0:
        return 1
    result = 1
```

```
    while N>0:
        result = (result * N) % MOD
        N-=3
    return result


# INPUT [uncomment & modify if required]

N = int(input())

# OUTPUT [uncomment & modify if required]
print(tripleFactorial(N))
```

Compilation Summary:

Compilation Status: **Compile Successfully** No Of Compilations: **1**

Defualt Input:

10

Candidate Output:

280

Test Case Summary:

Test Case: **1** Status: **Pass** Score:**0**

Test Case Input

```
10
```

Expected Output

280

Actual Output

280

Test Case: **2** Status: **Pass** Score:**2**

Test Case Input

```
4
```

Expected Output

4

Capgemini

Actual Output

4

Test Case: **3** Status: **Pass** Score:**2**

Test Case Input

0

Expected Output

1

Actual Output

1

Test Case: **4** Status: **Pass** Score:**2**

Test Case Input

8

Expected Output

80

Actual Output

80

Test Case: **5** Status: **Pass** Score:**2**

Test Case Input

9

Expected Output

162

Capgemini

Actual Output

162

Test Case: **6** Status: **Pass** Score:**2**

Test Case Input

17

Expected Output

209440

Actual Output

209440

Question: **#2**	Type: **Coding**	Skill: **Coding - Easy**	Status: **Answered**
Result: **Correct**	Level: **Easy**	Time Taken: **2 min 12 sec**	Average Time: **7 min 26 sec**
Score: **10 / 10**	Window Violation: **0 times**	Time Violation: **0 sec**	

Question #2

Minimum Amount to Spend

During the sale period, the shop owner considered an offer in which, on purchase of one item, another item of a price less than or equal to the purchased item would be given free of cost.
You are given an array of prices of ***N*** elements representing the price of ***N*** items.

Find the minimum amount to spend to buy all items.

Function Description
In the provided code snippet, implement the provided `minAmount(...)` method to find the minimum amount to spend to buy all items. You can write your code in the space below the phrase "`WRITE YOUR LOGIC HERE`".

There will be multiple test cases running, so the Input and Output should match exactly as provided.
The base Output variable `result` is set to a default value of `-404` , which can be modified.
Additionally, you can add or remove these output variables.

Input Format
The first line contains an integer ***N***, denoting the size of array ***A***.
The second line contains ***N*** space-separated integers, denoting the elements of the array ***A***.

Sample Input

```
6              -- denotes N
8 2 2 9 1 9    -- denotes A[i]
```

Constraints

1 <= ***N*** <= 1000

1 <= ***A[i]*** <= 10^4

Output Format

The output contains an integer denoting the minimum spending to buy all items.

Sample Output

```
19
```

Explanation

Prices ***A[i]*** = [8, 2, 2, 9, 1, 9]

On the purchase of one item, another item of a price less than or equal to the purchased item will be given free of cost.

So, we can buy an item at price ***9*** and get an item at price 9 for free.

Similarly, for an item priced at ***8*** and ***2***, we can get items priced 2 and 1 free of cost.

To do the same, we will sort the ***A[i]***.

After sorting, Prices (***A[i]***) = [1, 2, 2, 8, 9, 9].

So, the minimum amount to buy all the items = 9 + 8 + 2 = 19.

Hence, the output is ***19***.

Answer:

Coding Language: Python 3

Candidate Code:

```
def minAmount(N,A):
    #this is default OUTPUT. You can change it.
    result = -404
    A.sort(reverse=True)
    #write your Logic here:
    result = 0
    for i in range(0,N,2):
        result += A[i]




    return result

#INPUT [uncomment & modify if required]
N = int(input())
A = []
temp = input().split()
for i in range(N):
```


Capgemini

```
    A.append(int(temp[i]))
#OUTPUT [uncomment & modify if required]
print(minAmount(N,A))
```

Compilation Summary:

Compilation Status: **Compile Successfully** No Of Compilations: **1**

Defualt Input:

6

8 2 9 2 1 9

Candidate Output:

19

Test Case Summary:

Test Case: **1** Status: **Pass** Score:**0**

Test Case Input

```
6
8 2 9 2 1 9
```

Expected Output

19

Actual Output

19

Test Case: **2** Status: **Pass** Score:**2**

Test Case Input

```
10
3 8 1 1 9 2 2 3 3 3
```

Expected Output

18

Actual Output

18

Capgemini

Test Case: **3** Status: **Pass** Score:**2**

Test Case Input

```
10
2 64 10 19 34 64 64 64 9 10
```

Expected Output

181

Actual Output

181

Test Case: **4** Status: **Pass** Score:**2**

Test Case Input

```
8
1 1 1 1 4 4 4 4
```

Expected Output

10

Actual Output

10

Test Case: **5** Status: **Pass** Score:**2**

Test Case Input

```
7
4 4 2 1 99 471 99
```

Expected Output

575

Actual Output

Capgemini

575

Test Case: **6** Status: **Pass** Score:**2**

Test Case Input

```
133
9 8 2 3 4 6 5 1 9 8 2 2 3 3 7 2 10 5 9 9 3 10 7 3 6 10 3 4 9 1 7 8 7 5 9 1 3 7 10 2 9 6 8 1 10
 8 9 3 10 9 6 2 8 7 1 7 3 3 7 8 8 4 10 4 6 4 7 5 7 7 6 9 3 7 3 7 10 9 1 10 6 10 10 5 4 6 8 8 7
 8 2 3 1 5 2 3 3 1 10 1 9 1 4 9 8 6 4 8 7 3 9 2 9 1 9 2 9 10 3 5 5 1 9 9 1 9 6 4 2 8 2 9 2
```

Expected Output

384

Actual Output

384

Question: **#3**	Type: **Coding**	Skill: **Coding - Easy**	Status: **Answered**
Result: **Correct**	Level: **Easy**	Time Taken: **6 min 11 sec**	Average Time: **12 min 15 sec**
Score: **10 / 10**	Window Violation: **0 times**	Time Violation: **0 sec**	

Question #3

Number of Re-transmissions, Scheduling

A router has ***N*** transaction entries in the log.

The router sends these transaction entries to a server.

Each transaction has a ***transaction ID*** and ***transmission accuracy value.***

The transmission accuracy value varies from 0.1 to 1.0.

All the transaction IDs with transmission accuracy less than 0.5 should not be considered.

Print the transaction ID from the router log with the minimum transmission accuracy, or print -1 if there is no such transaction ID.

Note

Minimum transmission accuracy is defined as the smallest value of the given transmission accuracies (if a log contains 2 values with transmission accuracies of 0.6 and 0.3, the minimum transmission accuracy is 0.3).

Function Description

Capgemini

In the provided code snippet, implement the provided `findAccuracy(...)` method to find the transaction ID from the router log with the minimum transmission accuracy, or print -1 if there is no such transaction ID. You can write your code in the space below the phrase **"WRITE YOUR LOGIC HERE"**.

There will be multiple test cases running, so the Input and Output should match exactly as provided.
The base Output variable `result` is set to a default value of `-404` which can be modified. Additionally, you can add or remove these output variables.

Input Format

The first line contains an integer ***N***, denoting the number of transaction entries in the log of the router.
The next ***N*** subsequent lines contain 2 space-separated integers ***X*** and ***Y***, denoting the transaction ID and its corresponding transaction accuracy.

Sample Input

```
3        -- denotes N
1 0.3    -- denotes X and Y
2 0.5
3 0.7
```

Output Format

The output contains an integer denoting the transaction ID from the router log with the minimum transmission accuracy, or -1 if there is no such transaction ID.

Sample Output

```
2
```

Explanation

The minimum transmission accuracy required is 0.5 and above.
Transaction ID 2 is the ID that helps to obtain the minimum transmission accuracy of 0.5 above.
Hence, the output is ***2***.

Answer:

Coding Language: Python 3

Candidate Code:

```
def findAccuracy(N, log):
    #this is default OUTPUT. You can change it
    result = -404
    min_acc = float('inf')
    # WRITE YOUR LOGIC HERE:
    for i in range(N):
        txn_id = int(log[i][0])
        acc = log[i][1]
```

```
        if acc>=0.5 and acc<min_acc:
            min_acc = acc
            result = txn_id
    return result


# INPUT [uncomment & modify if required]
N = int(input())

log = [[0 for i in range(2)] for j in range(N)]

temp = []

for i in range(N):
    temp = input().split()

    for j in range(2):
        log[i][j] = float(temp[j])

# OUTPUT [uncomment & modify if required]
print(findAccuracy(N, log))
```

Compilation Summary:

Compilation Status: **Compile Successfully** No Of Compilations: **4**

Defualt Input:

3
1 0.3
2 0.5
3 0.7

Candidate Output:

```
2
```

Test Case Summary:

Test Case: **1** Status: **Pass** Score:**0**

Test Case Input

```
3
1 0.3
2 0.5
3 0.7
```

Expected Output

```
2
```


Capgemini

Actual Output

```
2
```

Test Case: **2** Status: **Pass** Score:**2**

Test Case Input

```
5
1 0.1
2 0.8
3 0.6
4 0.2
5 0.4
```

Expected Output

```
3
```

Actual Output

```
3
```

Test Case: **3** Status: **Pass** Score:**2**

Test Case Input

```
7
1 0.3
2 0.8
3 0.7
4 0.6
5 1.0
6 0.3
7 0.1
```

Expected Output

```
4
```

Actual Output

```
4
```

Test Case: **4** Status: **Pass** Score:**2**

Test Case Input

```
7
1 0.3
2 0.1
3 0.2
4 0.6
5 0.4
6 0.3
7 0.1
```

Expected Output

```
4
```

Actual Output

```
4
```

Test Case: **5** Status: **Pass** Score:**2**

Test Case Input

```
7
1 0.1
2 0.2
3 0.3
4 0.4
5 0.5
6 0.6
7 0.7
```

Expected Output

```
5
```

Actual Output

```
5
```

Test Case: **6** Status: **Pass** Score:**2**

Test Case Input

```
2
1 0.6
2 0.4
```


Capgemini

Expected Output

```
1
```

Actual Output

```
1
```

Question: **#4**	Type: **Coding**	Skill: **Coding - High**	Status: **Answered**
Result: **Correct**	Level: **Hard**	Time Taken: **32 min 41 sec**	Average Time: **12 min 53 sec**
Score: **16 / 20**	Window Violation: **0 times**	Time Violation: **0 sec**	

Question #4

Sorting Algorithms: Count Pick Ups

You have ***N*** cards lined up on a table, each with a number (not necessarily unique) written on it.

A special sorting method is used to sort them:

- The first card in the queue is picked up.
- If it is the card having the minimum number, it is removed from this queue.
- Otherwise, this card is put at the end of the queue.

The process ends when there is no card in the queue.

Find the number of times the first card is picked up from the queue.

Function Description

In the provided code snippet, implement the provided `countPickUps(...)` method to find the number of times the first card is picked up from the queue. You can write your code in the space below the phrase "`WRITE YOUR LOGIC HERE`".

There will be multiple test cases running, so the Input and Output should match exactly as provided.

The base Output variable `result` is set to a default value of `-404`, which can be modified. Additionally, you can add or remove these output variables.

Input Format

The first line contains an integer, denoting the number of cards ***N***.

The second line contains ***N*** space-separated integers, denoting the elements of the given array.

Sample Input

```
3          -- denotes N
1 3 2      -- denotes A[i]
```

Constraints

Capgemini

1 <= ***N***, ***A[i]*** <= 10^5

Output Format

The output contains a single integer, denoting the number of times the first card is picked up from the queue.

Sample Output

```
4
```

Explanation

N = 3
A[i] = [1, 3, 2]
1 is picked up. Since it is the minimum in the queue, this element is removed from the queue. The queue remains with [3, 2].
3 is picked up. Since it is not the minimum in the queue, the element is put at the back of the queue. The queue becomes [2, 3].
2 is picked up. Since it is the minimum in the queue, this element is removed from the queue. The queue remains with [3].
3 is picked up. Since it is the minimum in the queue, this element is removed from the queue. The queue becomes [].
The queue becomes empty.

The number of times the first card in the queue is picked up = 4.
Hence, the output is ***4***.

Answer:

Coding Language: Python 3

Candidate Code:

```
from collections import deque
def countPickUps(N, A):
    # this is default OUTPUT. You can change it.
    result = -404

    # write your Logic here:
    q=deque(A)
    sorted_A = sorted(A)
    min_index =0
    result=0
    while q:
        result+=1
        if q[0]==sorted_A[min_index]:
            q.popleft()
            min_index += 1
        else:
            q.append(q.popleft())

    return result


# INPUT [uncomment & modify if required]
```

```
N = int(input())

temp = input().split()
A = []
for i in range(N):
    A.append(int(temp[i]))

# OUTPUT [uncomment & modify if required]
print(countPickUps(N,A))
```

Compilation Summary:

Compilation Status: **Compile Successfully** No Of Compilations: **13**

Defualt Input:

3

1 3 2

Candidate Output:

```
4
```

Test Case Summary:

Test Case: **1** Status: **Pass** Score:**0**

Test Case Input

```
3
1 3 2
```

Expected Output

```
4
```

Actual Output

```
4
```

Test Case: **2** Status: **Pass** Score:**4**

Test Case Input

```
4
6 3 1 2
```

Expected Output

Capgemini

7

Actual Output

7

Test Case: **3** Status: **Pass** Score:**4**

Test Case Input

```
1
1000
```

Expected Output

1

Actual Output

1

Test Case: **4** Status: **Pass** Score:**4**

Test Case Input

```
5
3 3 3 3 3
```

Expected Output

5

Actual Output

5

Test Case: **5** Status: **Pass** Score:**4**

Test Case Input

```
5
4 41 44 45 44
```


Capgemini

Expected Output

6

Actual Output

6

Test Case: **6** Status: **Fail** Score:**0**

Test Case Input

```
6
4564 78787 454 12 454
```

Expected Output

16

Actual Output

Traceback (most recent call last):
File "main.py", line 29, in
A.append(int(temp[i]))
IndexError: list index out of range

Question: **#5**	Type: **Coding**	Skill: **Coding - Medium**	Status: **Answered**
Result: **Correct**	Level: **Medium**	Time Taken: **2 min 36 sec**	Average Time: **17 min 31 sec**
Score: **15 / 15**	Window Violation: **0 times**	Time Violation: **0 sec**	

Question #5

Sugarcane Parts

Alan has a sugarcane of length ***N***.

He divides the sugarcane into two equal parts:

If the length of the sugarcane is even, he divides the sugarcane into 3 equal parts such that the length of each part is a whole number (whole number: 1, 2, 3, 4, etc.).

Alan eats the part of the sugarcane which remains after dividing it into three parts.

Capgemini

For example, if the length of the sugarcane is 5, we can divide the sugarcane into 3 parts, each of length 1, and thus eats the sugarcane of length 2.

Find the total number of sugarcane parts at the end having a length of 1 unit.

Function Description

In the provided code snippet, implement the provided `sugarcaneParts(...)` method to find the total number of sugarcane parts at the end having a length of 1 unit. You can write your code in the space below the phrase `"WRITE YOUR LOGIC HERE"`.

There will be multiple test cases running so the Input and Output should match exactly as provided.

The base Output variable `result` is set to a default value of `-404` which can be modified. Additionally, you can add or remove these output variables.

Input Format

The first line contains an integer ***N***, denoting the length of the sugarcane.

Sample Input

```
5              -- denotes N
```

Constraints

1 <= ***N*** <= 1000

Output Format

The output contains a single integer denoting the total number of sugarcane parts at the end having a length of 1 unit.

Sample Output

```
3
```

Explanation

N = 5
As 5 is an odd number, we can divide it into {1, 1, 1}.
The length of all the parts is the same.
Alan eats the remaining part of length 2.
The total number of sugarcane parts at the end having a length of 1 unit = 3.
Hence, the output is ***3***.

Answer:

Coding Language: Python 3

Candidate Code:

Capgemini

```
def sugarcaneParts(N):
    # this is default OUTPUT. You can change it.
    result = -404

    # write your Logic here:
    if N==1:
        return 1
    if N%2==0:
        return 2 * sugarcaneParts(N//2)
    return 3*sugarcaneParts(N//3)

    return result


# INPUT [uncomment & modify if required]

N = int(input())

# OUTPUT [uncomment & modify if required]
print(sugarcaneParts(N))
```

Compilation Summary:

Compilation Status: **Compile Successfully** No Of Compilations: **1**

Defualt Input:

10

Candidate Output:

6

Test Case Summary:

Test Case: **1** Status: **Pass** Score:**0**

Test Case Input

```
10
```

Expected Output

6

Actual Output

6

Test Case: **2** Status: **Pass** Score:**3**

Test Case Input

Capgemini

```
1
```

Expected Output

```
1
```

Actual Output

```
1
```

Test Case: **3** Status: **Pass** Score:**3**

Test Case Input

```
11
```

Expected Output

```
9
```

Actual Output

```
9
```

Test Case: **4** Status: **Pass** Score:**3**

Test Case Input

```
110
```

Expected Output

```
108
```

Actual Output

```
108
```

Test Case: **5** Status: **Pass** Score:**3**

Test Case Input

```
1000
```


Capgemini

Expected Output

```
864
```

Actual Output

```
864
```

Test Case: **6** Status: **Pass** Score:**3**

Test Case Input

```
500
```

Expected Output

```
432
```

Actual Output

```
432
```

Question: **#6**	Type: **Coding**	Skill: **Exception Handling Coding**	Status: **Answered**
Result: **Correct**	Level: **Medium**	Time Taken: **9 min 39 sec**	Average Time: **12 min 31 sec**
Score: **10 / 10**	Window Violation: **0 times**	Time Violation: **0 sec**	

Question #6

Exception Handling: Range Sum

Harper Green has 2 integers, ***A*** and ***B***.
She must calculate the sum of all the integers between them (inclusive).
However, certain exceptional cases must be handled:

- If the start value is greater than the end value, throw a "**Start Greater Than End**" Exception.
- If the start value is equal to the end value, throw a "**Start Equals To End**" Exception.
- If any of the input values are negative, throw a "**Negative Number**" Exception.

Print the sum of all the integers in the given range.

Function Description

In the provided code snippet, implement the provided `rangesum(...)` method to print the sum of all

Capgemini

the integers in the given range. You can write your code in the space below the phrase "`WRITE YOUR LOGIC HERE`".

There will be multiple test cases running so the Input and Output should match exactly as provided.
The base Output variable `result` is set to a default value of `-404` which can be modified. Additionally, you can add or remove these output variables.

Input Format
The first line contains a single integer ***A***, denoting the start value.
The second line contains a single integer ***B***, denoting the end value.

Sample Input

```
2  -- denotes A
6  -- denotes B
```

Constraints
-1000 ≤ ***A***, ***B*** ≤ 1000

Output Format
The output contains a single integer denoting the sum of all the integers in the given range.

Sample Output

```
20
```

Explanation
A = 2
B = 6
The function should calculate the sum of all the integers between 2 and 6 (inclusive).
The sum is 2 + 3 + 4 + 5 + 6 = 20.
Hence, the output is ***20***.

Answer:

Coding Language: Python 3

Candidate Code:

```
def rangesum(A, B):
    # This is the default output. You can change it.
    result = -404
    # Write your logic here:
    if A<0 or B<0:
        return "Negative Number"
    if A > B:
        return "Start Greater Than End"
    if A==B:
        return "Start Equals To End"
    result = 0
    for i in range(A,B+1):
```

```
        result += i

    return result

#INPUT [uncomment & modify only if required]
A = int(input())
B = int(input())

#OUTPUT [uncomment & modify only if required]
print(rangesum(A, B))
```

Compilation Summary:

Compilation Status: **Compile Successfully** No Of Compilations: **5**

Defualt Input:

2

6

Candidate Output:

20

Test Case Summary:

Test Case: **1** Status: **Pass** Score:**0**

Test Case Input

2
6

Expected Output

20

Actual Output

20

Test Case: **2** Status: **Pass** Score:**2**

Test Case Input

12
9

Expected Output

Capgemini

Start Greater Than End

Actual Output

Start Greater Than End

Test Case: **3** Status: **Pass** Score:**2**

Test Case Input

```
-12
13
```

Expected Output

Negative Number

Actual Output

Negative Number

Test Case: **4** Status: **Pass** Score:**2**

Test Case Input

```
200
200
```

Expected Output

Start Equals To End

Actual Output

Start Equals To End

Test Case: **5** Status: **Pass** Score:**2**

Test Case Input

```
7
15
```


Capgemini

Expected Output

```
99
```

Actual Output

```
99
```

Test Case: **6** Status: **Pass** Score:**2**

Test Case Input

```
20
50
```

Expected Output

```
1085
```

Actual Output

```
1085
```

Disclaimer

This report is generated electronically based on the information provided by the assessment participants. It also includes flags, especially when proctoring services are utilized. However, it is important to emphasize that this report should not be the sole basis for making any business, selection, entrance, or employment-related decisions. iMocha assumes no responsibility for any consequences arising from the use of this report or any actions taken or refrained from as a result of it. This includes all business decisions made in reliance on the information, advice, or AI flags contained in this report, as well as any sources of information mentioned or referred to in the report.